

Homework 5

1 Job Fair Speedrun

1.1 a

smallest acceptable set of attendance times for the example in Fig1: 3

1.2 b

counterexample to find the minimum hitting set counted by always picking the time that covers the largest number of remaining employers:

Time	0	1	2	3	4	5	6	7	8	9
A	•	—	•							
B		•	—	—	—	•				
C		•	—	—	—	•				
D					•	—	—	—	•	
E					•	—	—	—	•	
F								•	—	•

Greedy picks $t = 4$, covering B, C, D, and E (the maximum overlap of four). A and F require two additional visits, for three visits total. The optimal solution uses only $t = 1$ and $t = 8$, covering all six intervals.

1.3 c

If the input contains k mutually disjoint intervals, then every acceptable set of visit time must have size at least k . Proof by contradiction

1.3.1 Claim

The minimum requirement for an *acceptable* set of time visit is that each employer's interval $[s_i, f_i]$ contains at least one t_j such that $s_i \leq t_j \leq f_i$.

1.3.2 Proof by Contradiction

Suppose if the input contains k mutually disjoint intervals, then every acceptable set of visit times has size less than k .

If every interval is mutually disjoint, then $f_{i-1} < s_i$ and $f_i < s_{i+1}$ such that $[s_{i-1}, f_{i-1}] \cap [s_i, f_i] \cap [s_{i+1}, f_{i+1}] = \emptyset$.

So, t_j that $s_i \leq t_j \leq f_i$ cannot exist in neither $[s_{i-1}, f_{i-1}]$ nor $[s_{i+1}, f_{i+1}]$ such that there must exist at least one visit time t for each interval for mutually disjoint set.

Therefore, if there are k mutually disjoint intervals, then the least number of t must be k . This contradicts the supposition.

QED.

1.4 d

Algorithm 1: JFS(timetable: list[list[int,int]]) $\rightarrow$ list[int]

```

1 Interval <- empty Priority Queue with minheap of ascending finishing time
2 // set time interval as priority queue with minheap of earliest finishing time
3 for i in range(timetable.length):
4     Interval.enqueue(timetable[i])
5 opt_interval <- empty nested list
6 min <- 0
7 while !Interval.isempty():
8     opt_interval[min][0] <- Interval.extract_min() // if the PQ is not empty, then
        there is always the leftover. initialize the interval list with the first
        interval
9     count <- 1 // counter for optimizing the number of visit times; counting the
        overlaps
10    while !Interval.isempty():
11        if Interval.get_min()[0] > opt_interval[min][0][1]: // if s_{i+count} ≤
            f_{i}, then overlap. Otherwise, disjoint.
12            break
13        opt_interval[min][count] <- Interval.extract_min() // we remove the previous
            compared interval; the final length of opt_interval will be the
            minimum(optimized) number of visit times
14        count <- count + 1
15    min <- min + 1
16 acceptable <- empty list[int]
17 for i in range(opt_interval.length):
18     acceptable[i] <- intersection(opt_interval[i])
19 return acceptable

```

Algorithm 2: intersection(opt_interval_i)

```

1 intersect <- opt_interval_i[0][0] // initialize with first element
2 for i = 1; i < opt_interval_i.length; i++ {
3     intersect <- max(intersect, opt_interval_i[i][0])
4 }
5 // intersection among the overlaps: maximum starting time among the intervals. if
    that equals the finish time of the first interval, be that as the intersection
    time.
6 return intersect

```

1.5 e

1. $O(\log n)$: Priority Queue enqueue takes $O(\log n)$ average and worst case for each. upper bound is $O(n \log n)$.
2. $O(n \log n)$: we loop through all of the nodes in the Priority Queue once so that it iterates n times. However, as we extract minimum from the minheap, we have to reallocate the nodes in ascending finishing time that takes $\log n$ times.

3. $O(n)$: getting acceptable times from the minimum(optimal) time visit intersections takes n times worst case.

$O(\log n) + O(n \log n) + O(n) = O(n \log n)$. Therefore, the total runtime of the algorithm is $O(n \log n)$.

1.6 f

Proof of Correctness To prove the correctness of the algorithm:

1. There must be at least one acceptable visit time for each interval.
2. set of acceptable visit times must be in minimum size
3. the acceptable set of visit times must cover all time intervals given as input.

First, the algorithm initializes the priority queue Interval with the first element of leftover time interval for every outer iteration. Then, we assign the optimal time visit for every group of time intervals that can have an acceptable visit time. Because each assigned optimal time visit must intersect all of the time intervals within the same group, there is at least one acceptable visit time for each interval. This leads to the conclusion that the acceptable set of visit times covers all groups of the time intervals given as input.

Moreover, according to the lower bound of the set of acceptable visit times, the algorithm must output k visit times for k mutually disjoint intervals. the inner loop of while ensures that the groups get separated by disjoints such that there is a mutual disjoint between the groups. Therefore, by the lower bound proven at part (c), the minimum number of visit time must be the length of opt_interval. Iterating by the length of opt_interval to get the visit times, the algorithm ensures that there are only minimum number of visit times in the output.

Therefore, the algorithm output is always correct.

2 Earliest Interval

2.1 Pseudocode

Algorithm 3: EarlyInterval($T: \text{list}[\text{list}[\text{int}, \text{int}]] \rightarrow \text{list}[\text{int}]$)

```
1 /* Goal: for each finish time, get the earliest-starting interval that contains
   that finish time. ( $f_k \in [s_j, f_j]$ ). output a list of  $n$  indices such that for
   the  $k$ th finish time,  $A[k]$  is the value of the index asked to find
2 T: list of intervals ( $[s_i, f_i]$ )
3 */
4 EFT <- EFT(range(T.length)) // sort T's intervals' indices by EFT first.
   $Theta(n log n)$
5 EIList <- initialize Priority Queue with minheap ordered by EST
6 for priorityidx in range(T):
7   EIList.insert(priorityidx, T[priorityidx][0]) // id, key pair for Priority
   Queue; id: priority integer, key: starting times
8 /* now we have a priority queue sorted by EST(EIList) and a nested list sorted by
   EFT(EFT)*/
9 A <- empty index list
10 // this iterates  $O(n)$  worst time
11 for k in range(EFT.length):
12   i <- EFT[k] // we need to iterate through EFT interval to find what intervals
   belong to which each EST interval
13   f <- T[i][1] // finish time to compare
14   while T[EIList.get_min()[0]][1] < f: // while finish time of EST is less than
   the EFT finish time; if finish time of EST is less than the EFT finish time,
   then that EST interval will never be used to entail the finish time of any
   interval so we subtract his ass.
15     EIList.extract_min() // if the condition met, we are sure that this belongs
   to no interval so out
16   A[i] <- EIList.get_min()[0] // update priority index that satisfy the problem
17 return A
```

2.2 Runtime Analysis

1. $O(n \log n)$: sorting by Earliest Finishing Time takes $n \cdot \log n$ times as per the lecture.
2. $O(n \log n)$: T is a list of intervals that each interval is a pair of start and finish time. There are n intervals with $O(\log n)$ for insertion in the priority by EST
3. $O(n \log n)$: main loop to get A . the outer for loop iterates n times. inner while loop removes n intervals at worst case only once. and each `extract_min()` operation would cost $O(\log n)$.

$O(n \log n) + O(n \log n) + O(n \log n) = O(n \log n)$ Therefore. the total runtime of the algorithm is $O(n \log n)$.

2.3 Proof of Correctness

To prove the correctness of the algorithm:

1. $s_j \leq f_k \leq f_j$
2. for every interval cmp that contains f_k , $s_j \leq s_{cmp}$.

First, we prepared three different intervals: T : original list of intervals, EFT : intervals by earliest finish time, $EIList$: intervals by earliest start time. By definitions of EST, the first interval of EST will be assigned to each finish time as long as that finish time is less than the finish time of first interval. By definitions of EFT, the finish time of the following interval will never be within the range of its preceeding interval as we assume each start and finish is distinct.

With these properties, if finish time of EST is less than the EFT finish time, then that EST interval will never be used to entail the finish time of any interval. The algorithm ensures that point with while inner loop and assigning the leftover EST interval index to A . By repeating this operation for the entire n intervals, we can ensure both $s_j \leq f_k \leq f_j$ and for every interval cmp that contains f_k , $s_j \leq s_{cmp}$.

Therefore, the alogrithm always outputs correctly.